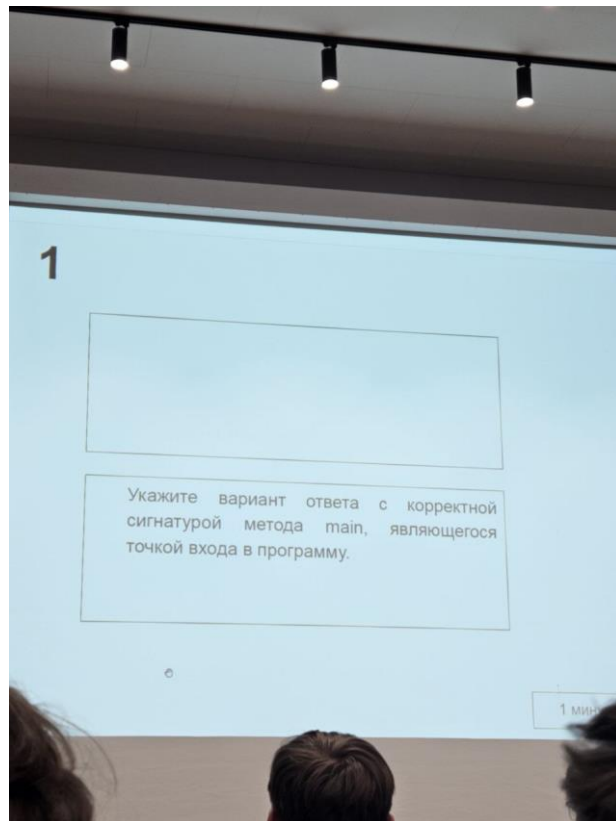
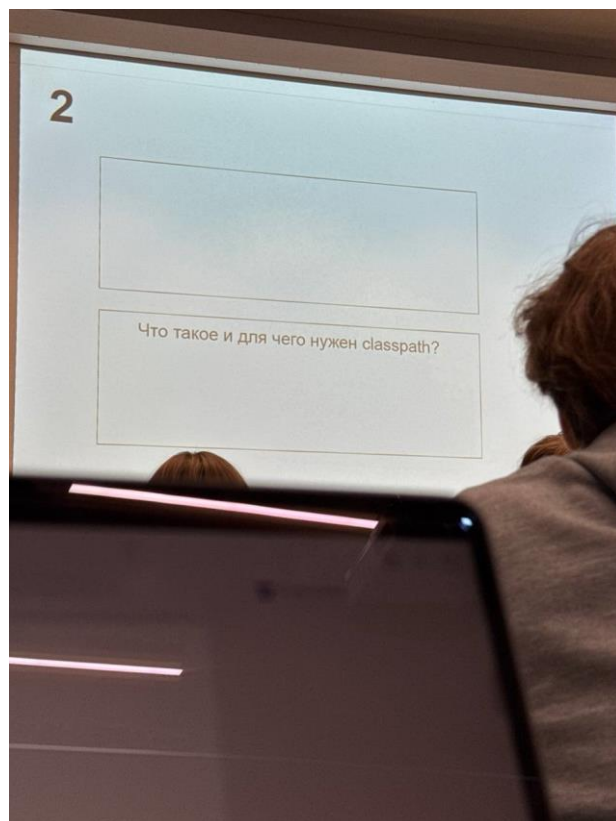
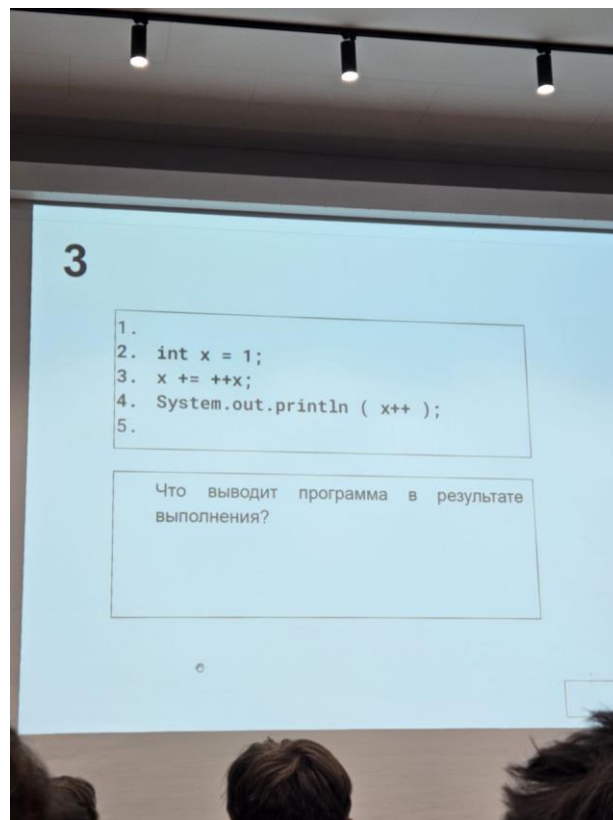




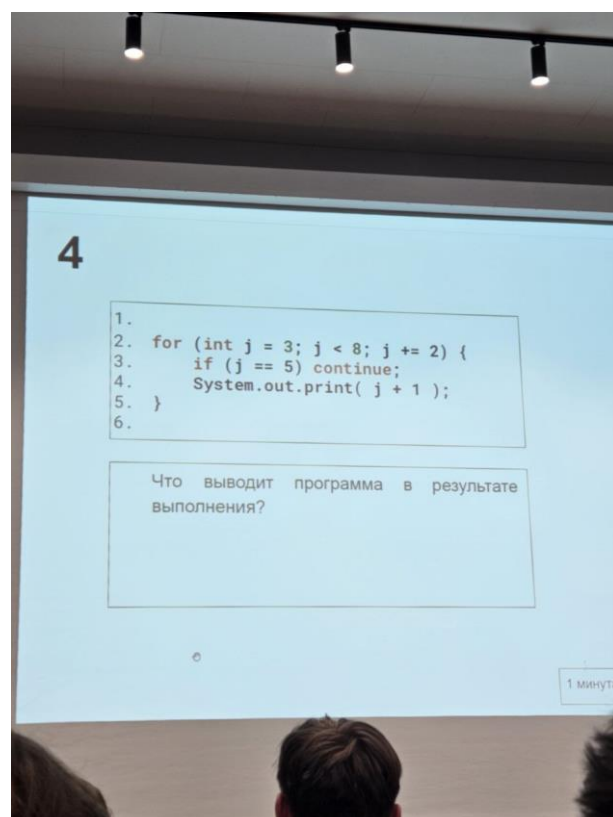
Ответ: `public static void main(String[] args)`



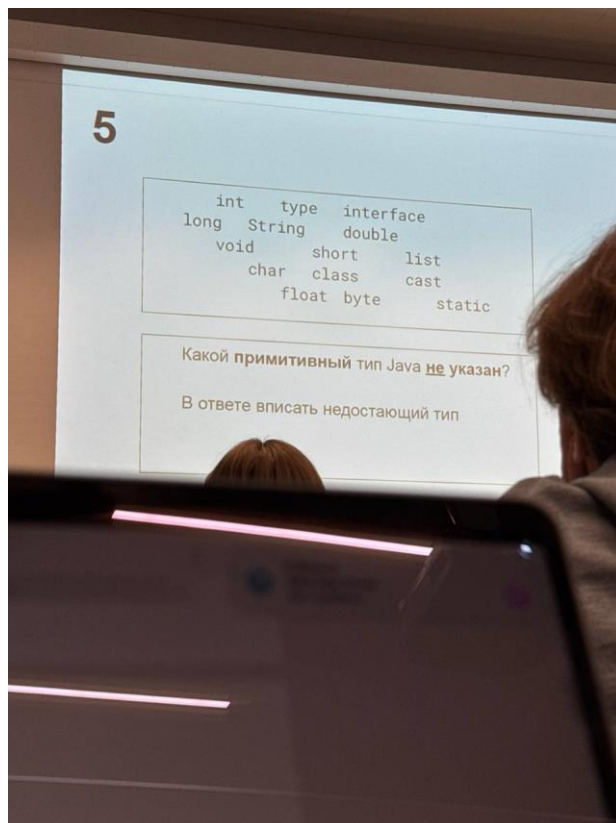
Ответ: Это строка, указывающая расположение файлов .class (собранных Java классов), для того чтобы JVM и javac смогли выполнить программу



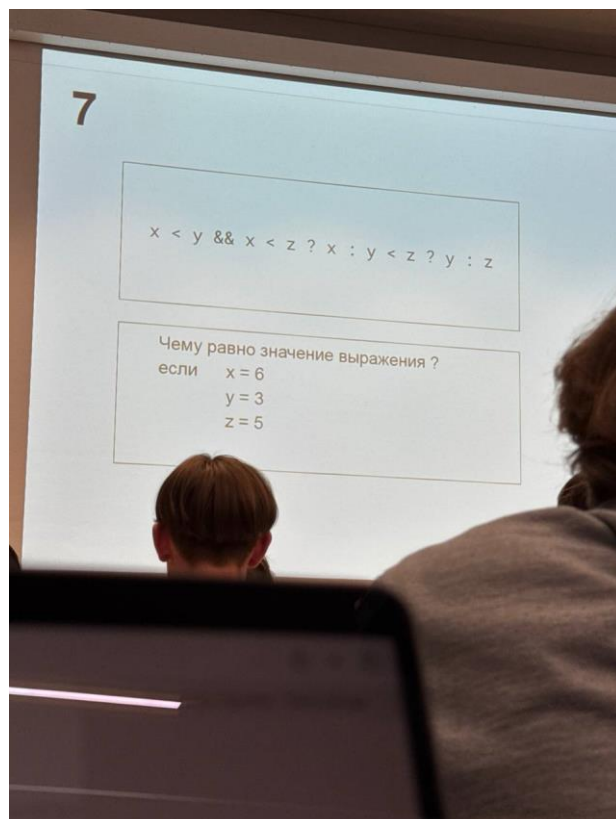
Ответ: 2



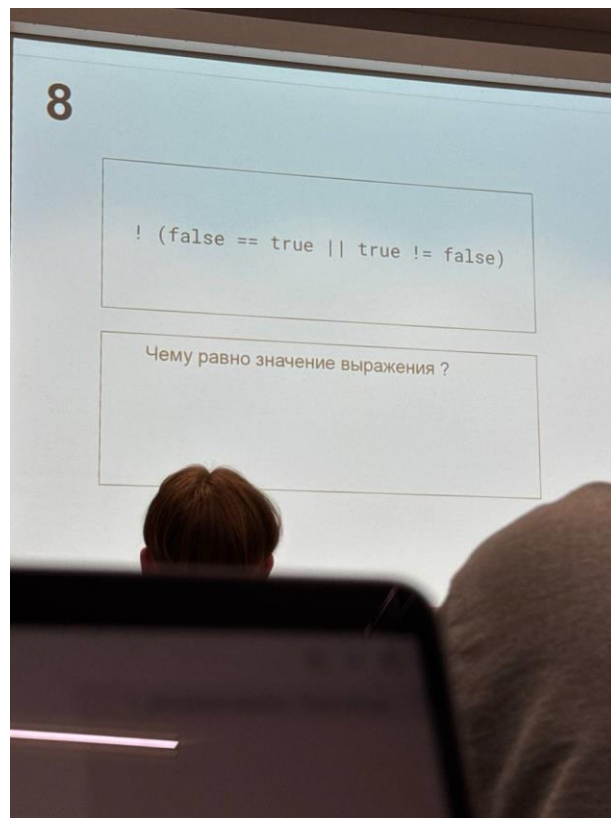
Ответ: 4; 8



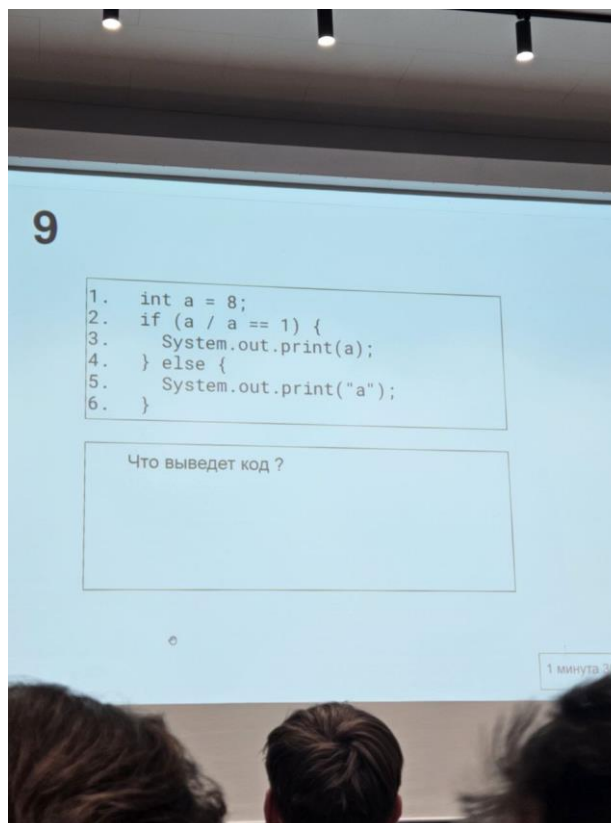
Ответ: boolean



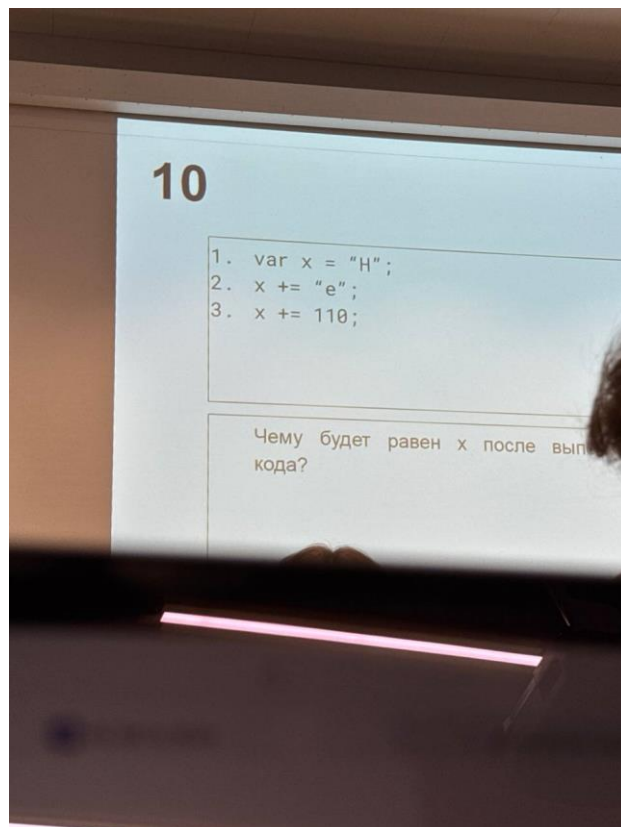
Ответ: 3



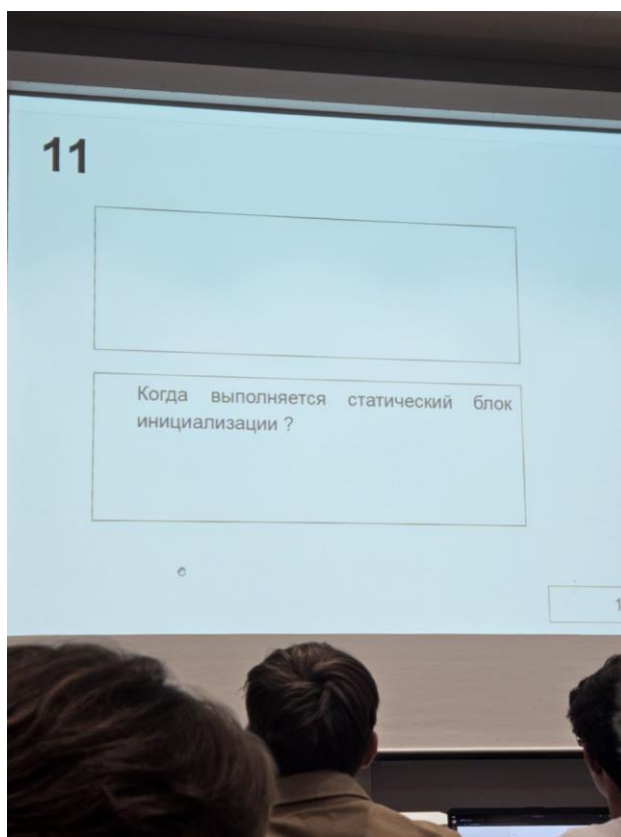
Ответ: false



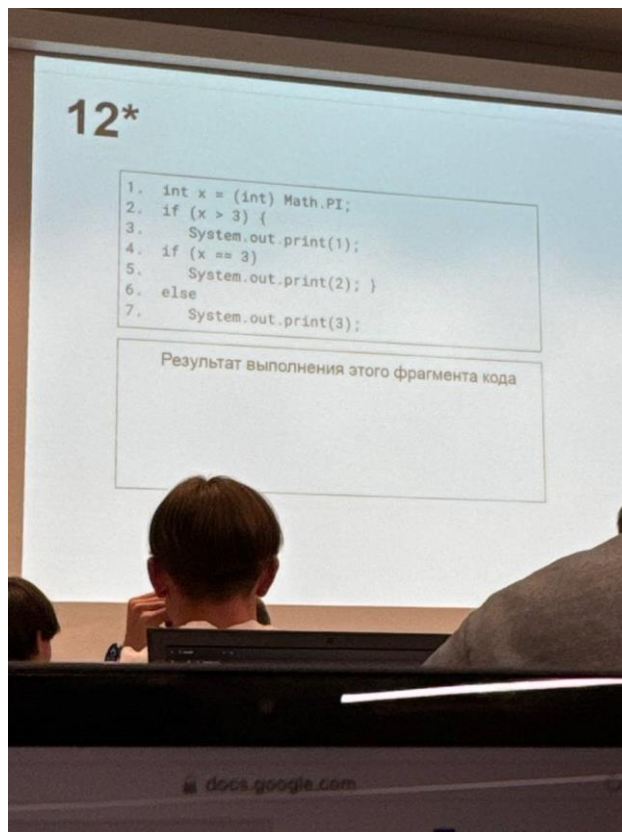
Ответ: 8



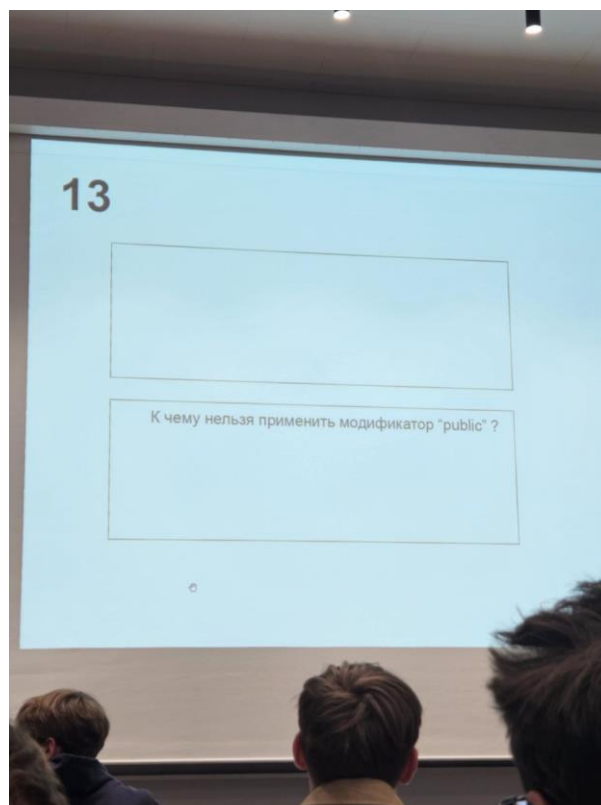
Ответ: He110



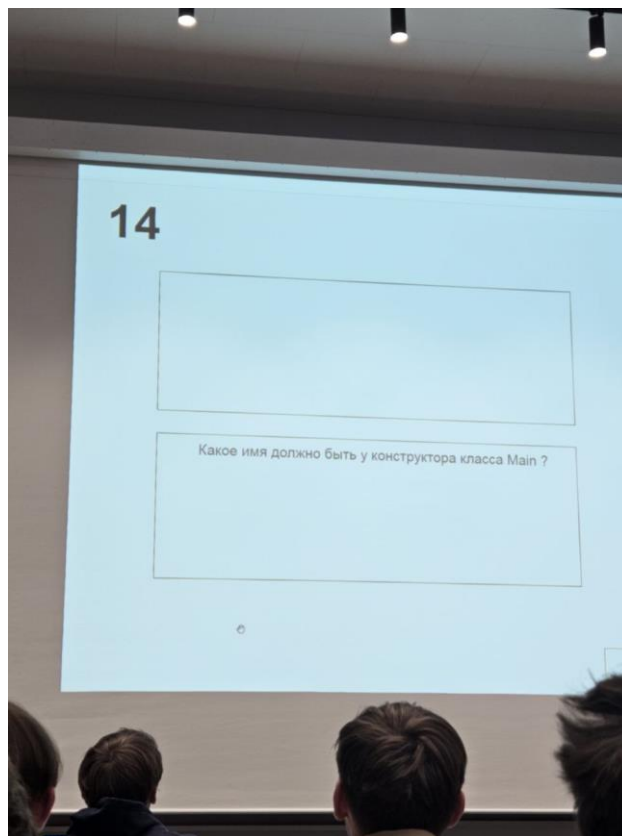
Ответ: выполняется один раз, когда класс загружается в память.



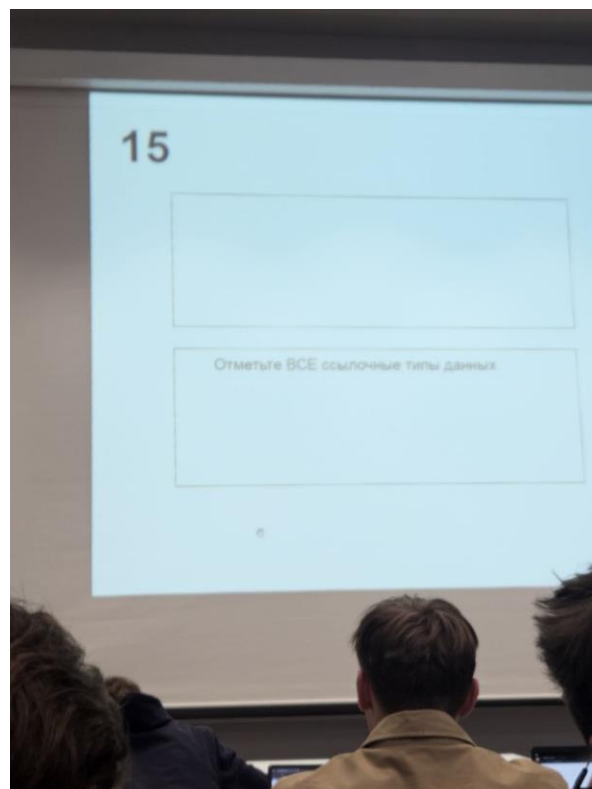
Ответ: 2



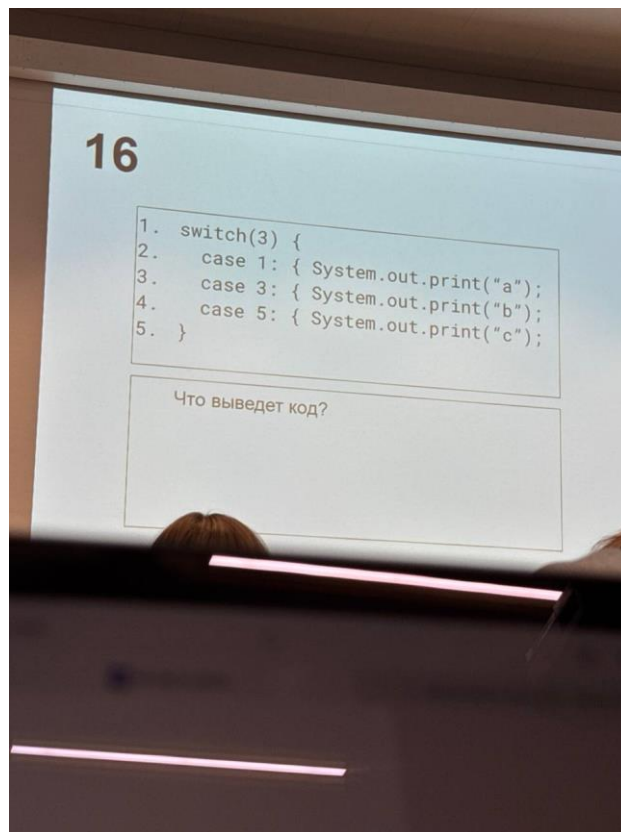
Ответ: Для локальных переменных в методах; Для локальных классов в методах; Для конструкторов в интерфейсах.



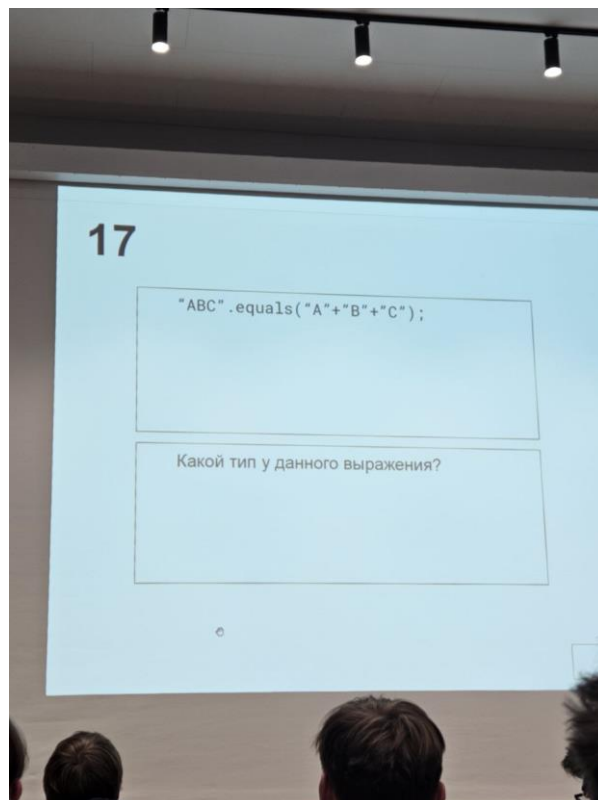
Ответ: Main



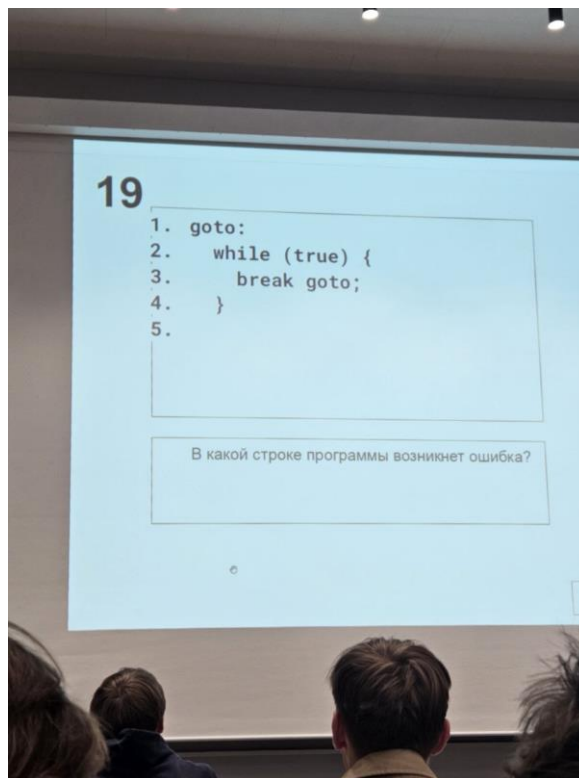
Ответ: Мы знаем пока 4:
Классы; Интерфейсы; Массивы; Строки



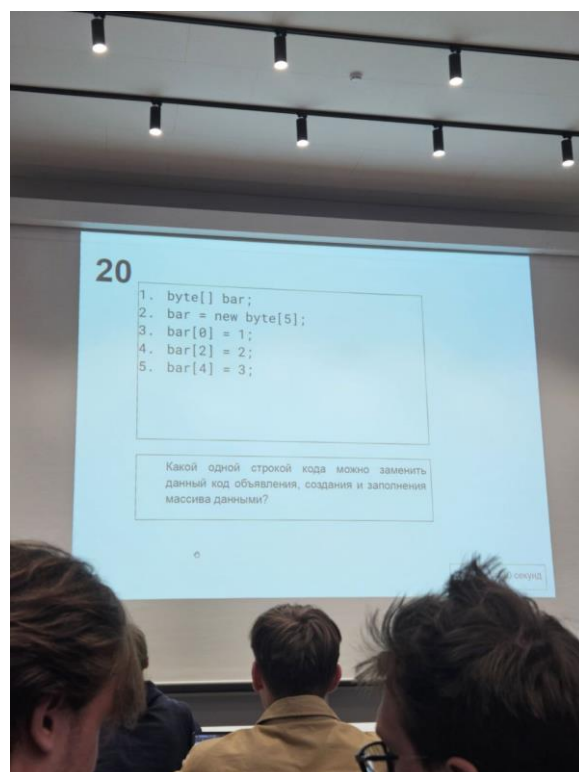
Ответ: b; c



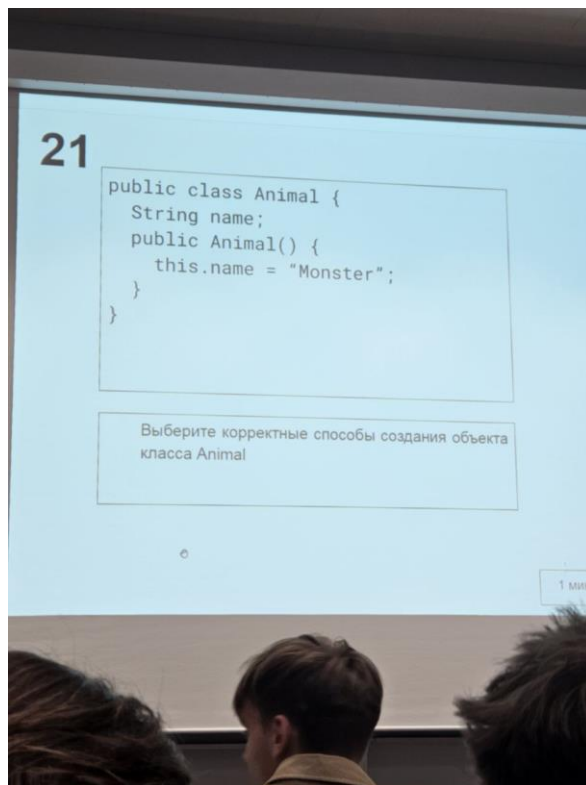
Ответ: boolean



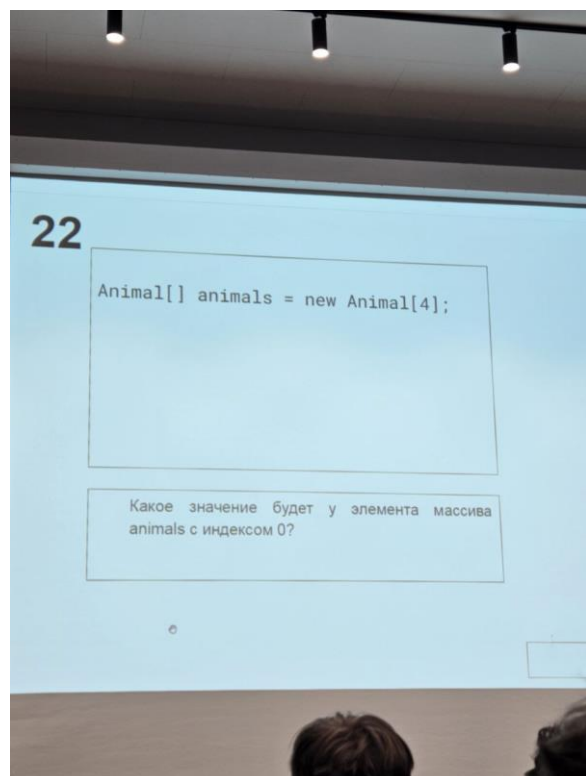
Ответ: Если вопрос про первую ошибку, то “1”, если все ошибки, то “1”, “3”
“НЕ УВЕРЕН В ОТВЕТЕ”



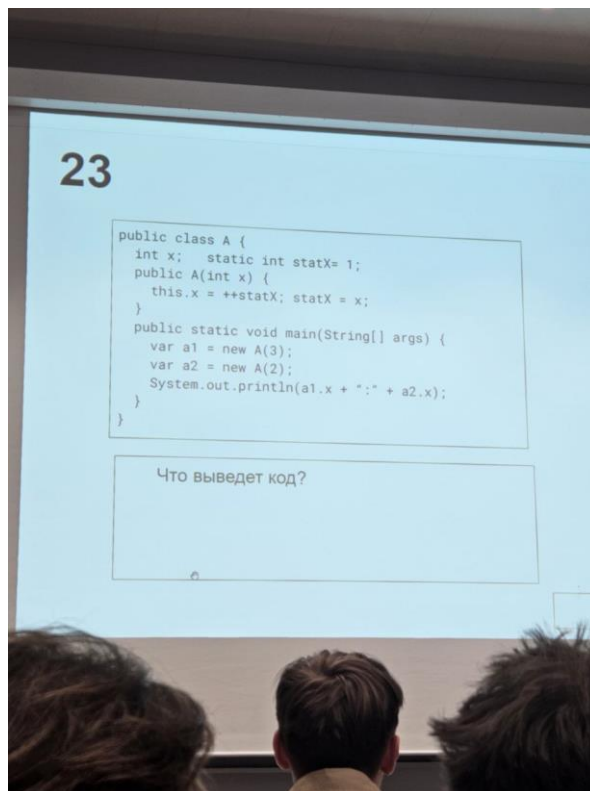
Ответ: `byte[] bar = {1, 0, 2, 0, 3};`



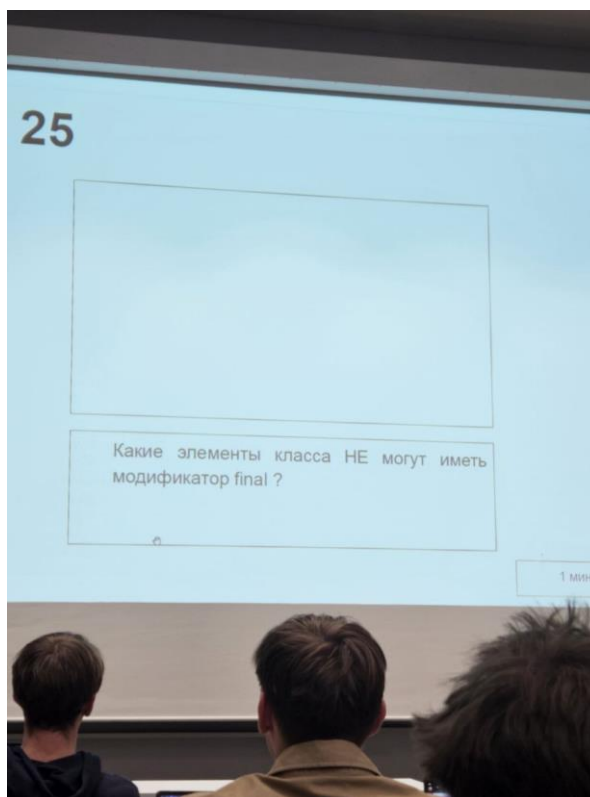
Ответ: `Animal animals = new Animal();`



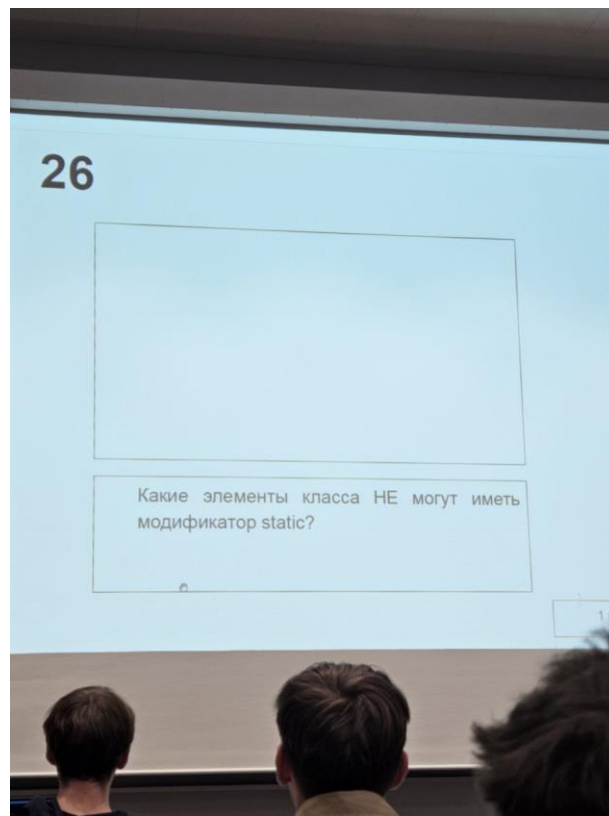
Ответ: `null`



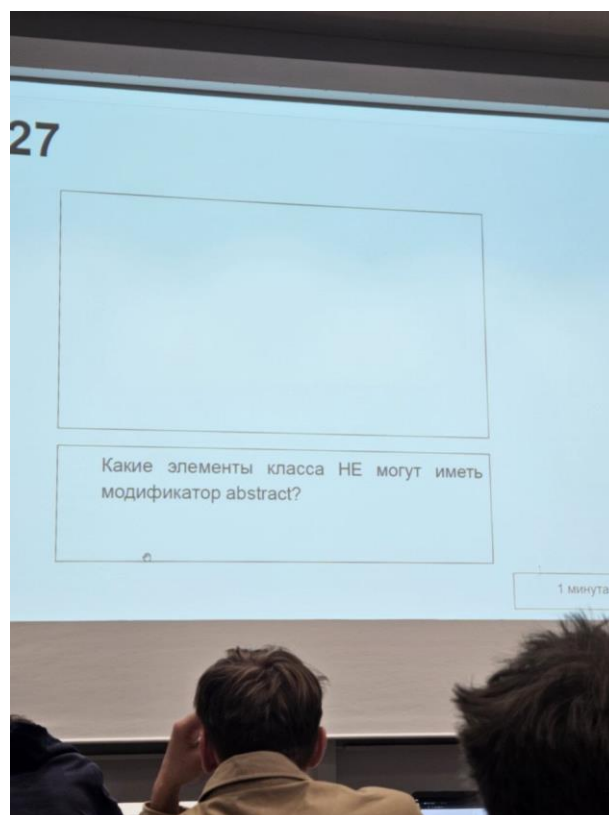
Ответ: 2:4



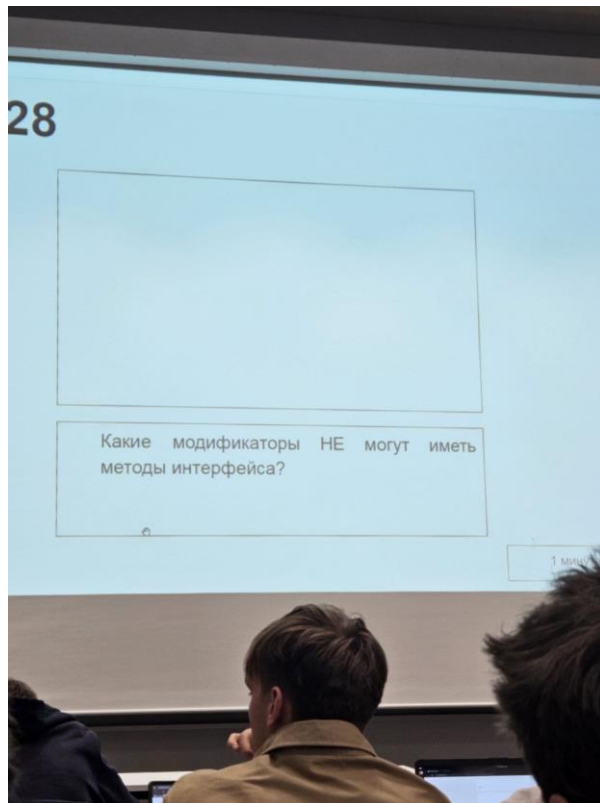
Ответ: Локальные переменные в методах; Поля внутри интерфейсов;
Вложенный класс



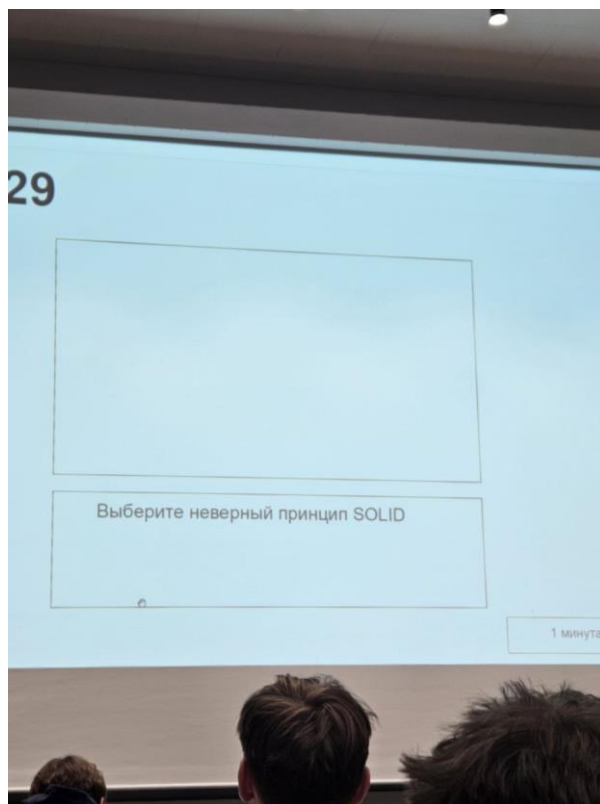
Ответ: Локальные переменные в методах; Абстрактные методы; Вложенные классы



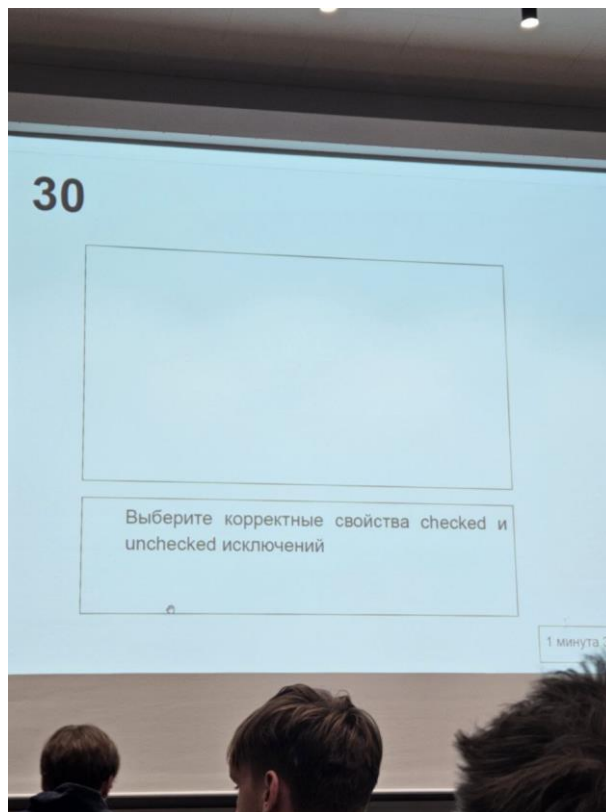
Ответ: static/private/final методы; Поля



Ответ: private; protected; abstract



Ответ: Dependency Injection Principle



Ответ:

30

- ☐ Проверяемые исключения проверяет компилятор, поэтому их можно не обрабатывать
- ☒ Компилятор заставляет обработать проверяемые исключения или объявить их в сигнатуре метода
- ☐ Непроверяемые исключения нельзя обработать с помощью блока try-catch
- ☒ Проверяемые исключения возникают на этапе компиляции программы
- ☒ Непроверяемые исключения возникают во время выполнения программы
- ☐ Если любые исключения не обработаны, программа не запустится